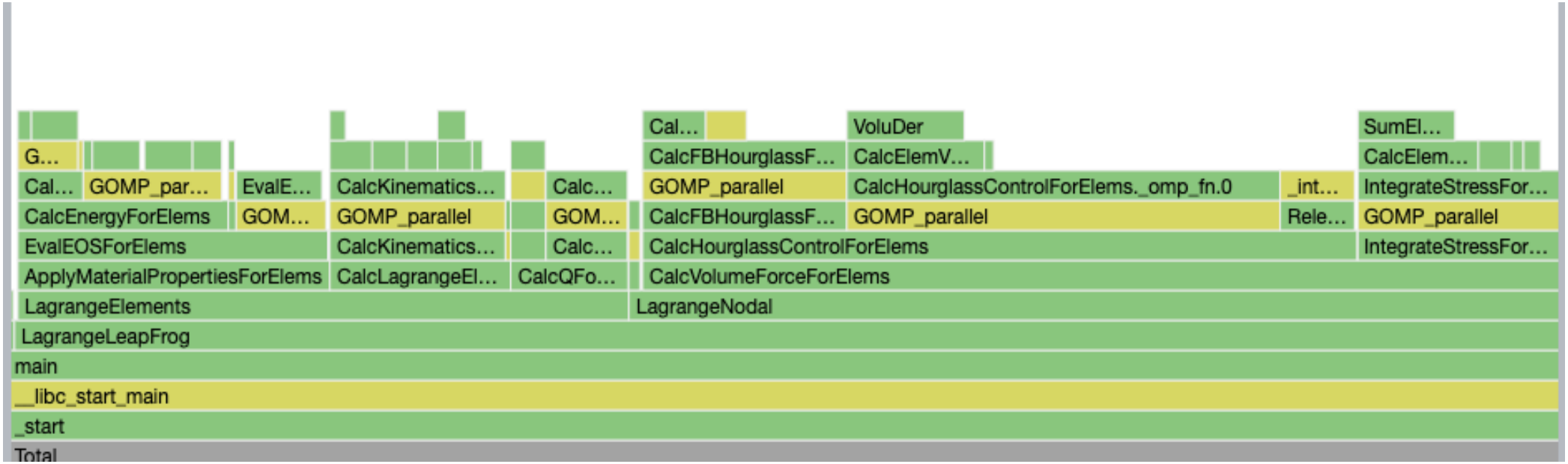
OMP_NUM_THREADS=1



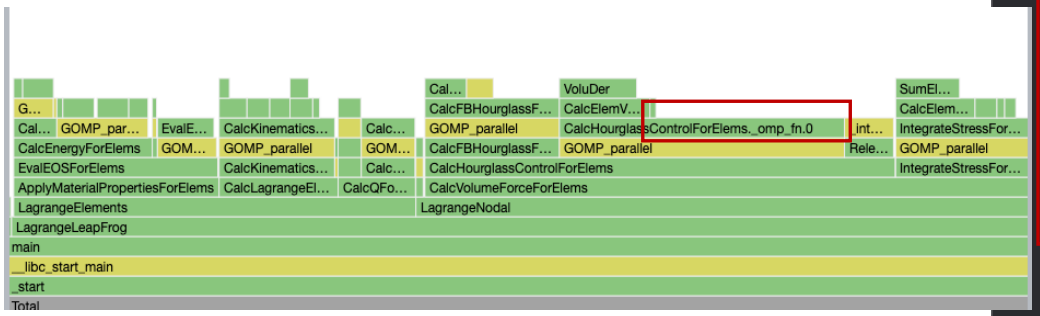
Analysis Configuration | Collection Log | Summary | Bottom-up | Caller/Callee | Top-down Tree | Flam

Function	CPU Time: Total	CPU Time: Self
CalcHourglassControlForElems._omp_fn.0	27.9%	4.390s
VoluDer	7.6%	1.798s
CalcFBHourglassForceForElems._omp_fn.0	13.2%	1.541s
SumElemFaceNormal	6.3%	1.489s
EvalEOSForElems._omp_fn.0	5.5%	1.305s
CalcMonotonicQGradientsForElems._omp_fn.0	5.3%	1.250s
_int_free	4.8%	1.150s
CalcElemShapeFunctionDerivatives	4.4%	1.040s
CalcElemFBHourglassForce	4.1%	0.970s
CalcEnergyForElems._omp_fn.3	3.1%	0.737s
CalcEnergyForElems._omp_fn.1	3.1%	0.728s
CalcPressureForElems._omp_fn.1	3.0%	0.715s
cbrtf64	2.6%	0.620s
CalcMonotonicQRegionForElems._omp_fn.0	2.2%	0.530s
CollectDomainNodesToElemNodes	2.0%	0.480s
CalcElemVelocityGradient	2.0%	0.470s

1. CalcHourglassControlForElems._omp_fn.0

(27.9% = 6.6s)

(4.3s actually inside the function)

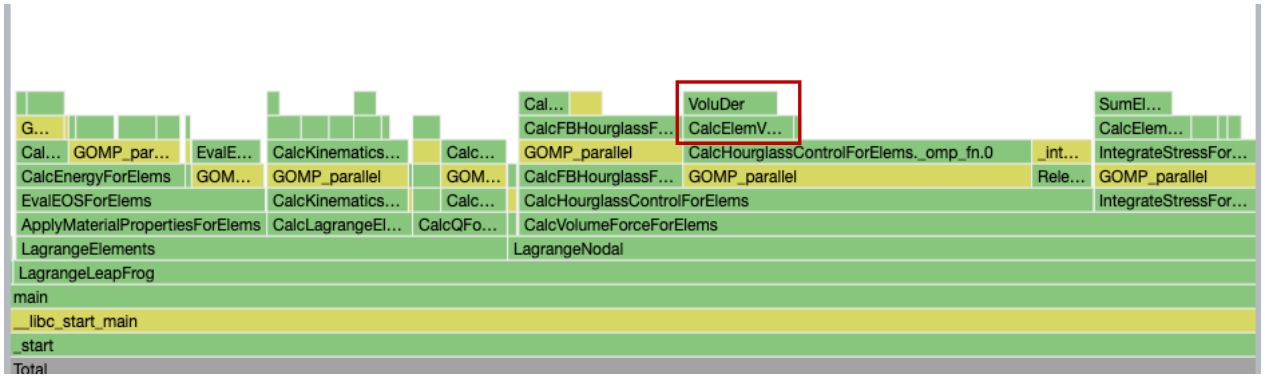



```
   /* start loop over elements */
#pragma omp parallel for firstprivate(numElem)
   for (Index_t i=0 ; i<numElem ; ++i){
      Real_t  x1[8],  y1[8],  z1[8] ;
      Real_t pfx[8], pfy[8], pfz[8] ;

      Index_t* elemToNode = domain.nodelist(i);
      CollectDomainNodesToElemNodes(domain, elemToNode, x1, y1, z1);

      CalcElemVolumeDerivative(pfx, pfy, pfz, x1, y1, z1);

      /* load into temporary storage for FB Hour Glass control */
      for(Index_t ii=0;ii<8;++ii){
         Index_t jj=8*i+ii;

         dvdx[jj] = pfx[ii];
         dvdy[jj] = pfy[ii];
         dvdz[jj] = pfz[ii];
         x8n[jj]  = x1[ii];
         y8n[jj]  = y1[ii];
         z8n[jj]  = z1[ii];
      }

      determ[i] = domain.volo(i) * domain.v(i);

      /* Do a check for negative volumes */
      if ( domain.v(i) <= Real_t(0.0) ) {
#if USE_MPI
         MPI_Abort(MPI_COMM_WORLD, VolumeError) ;
#else
         exit(VolumeError);
#endif
      }
   }
```

2. VoluDer (7.6%)

<- CalcElemVolumeDerivative (8.9%)

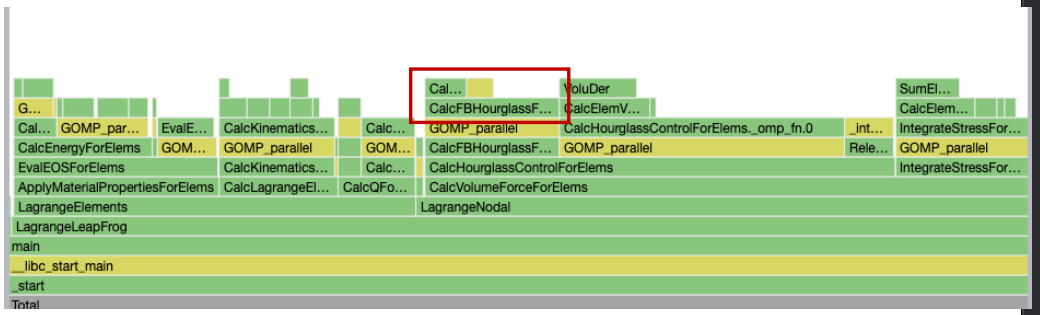
<- 1 CalcHourglassControlFroElems._omp_fn.0 (27.9%)




```c
static inline
void CalcElemVolumeDerivative(Real_t dvdx[8],
                              Real_t dvdy[8],
                              Real_t dvdz[8],
                              const Real_t x[8],
                              const Real_t y[8],
                              const Real_t z[8])
{
   VoluDer(x[1], x[2], x[3], x[4], x[5], x[7],
           y[1], y[2], y[3], y[4], y[5], y[7],
           z[1], z[2], z[3], z[4], z[5], z[7],
           &dvdx[0], &dvdy[0], &dvdz[0]);
   VoluDer(x[0], x[1], x[2], x[7], x[4], x[6],
           y[0], y[1], y[2], y[7], y[4], y[6],
           z[0], z[1], z[2], z[7], z[4], z[6],
           &dvdx[3], &dvdy[3], &dvdz[3]);
   VoluDer(x[3], x[0], x[1], x[6], x[7], x[5],
           y[3], y[0], y[1], y[6], y[7], y[5],
           z[3], z[0], z[1], z[6], z[7], z[5],
           &dvdx[2], &dvdy[2], &dvdz[2]);
   VoluDer(x[2], x[3], x[0], x[5], x[6], x[4],
           y[2], y[3], y[0], y[5], y[6], y[4],
           z[2], z[3], z[0], z[5], z[6], z[4],
           &dvdx[1], &dvdy[1], &dvdz[1]);
   VoluDer(x[7], x[6], x[5], x[0], x[3], x[1],
           y[7], y[6], y[5], y[0], y[3], y[1],
           z[7], z[6], z[5], z[0], z[3], z[1],
           &dvdx[4], &dvdy[4], &dvdz[4]);
   VoluDer(x[4], x[7], x[6], x[1], x[0], x[2],
           y[4], y[7], y[6], y[1], y[0], y[2],
           z[4], z[7], z[6], z[1], z[0], z[2],
           &dvdx[5], &dvdy[5], &dvdz[5]);
   VoluDer(x[5], x[4], x[7], x[2], x[1], x[3],
           y[5], y[4], y[7], y[2], y[1], y[3],
           z[5], z[4], z[7], z[2], z[1], z[3],
           &dvdx[6], &dvdy[6], &dvdz[6]);
   VoluDer(x[6], x[5], x[4], x[3], x[2], x[0],
           y[6], y[5], y[4], y[3], y[2], y[0],
           z[6], z[5], z[4], z[3], z[2], z[0],
           &dvdx[7], &dvdy[7], &dvdz[7]);
}
```

3. CalcFBHourglassForceForElems._omp_fn.0 (13.2%)

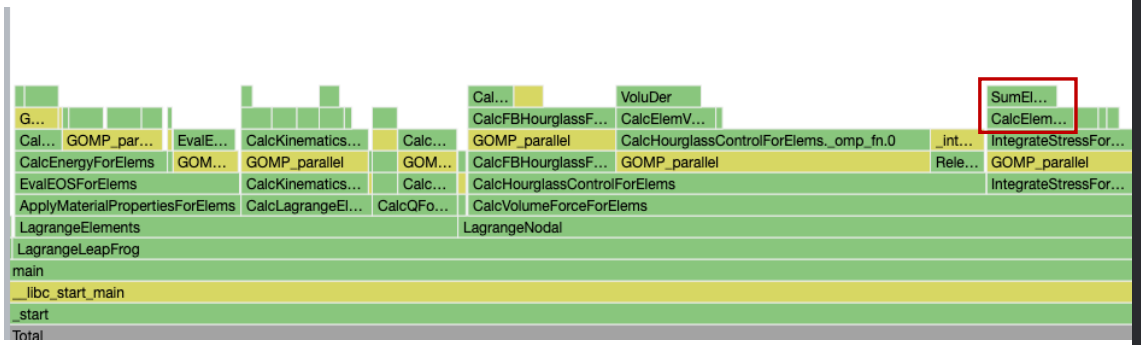
Only calls CalcElemFBHourglassForce which just 6 loops




```
#pragma omp parallel for firstprivate(numElem, hourg)
   for(Index_t i2=0;i2<numElem;++i2){
      Real_t *fx_local, *fy_local, *fz_local ;
      Real_t hgfx[8], hgfy[8], hgfz[8] ;

      Real_t coefficient;

      Real_t hourgam[8][4];
      Real_t xd1[8], yd1[8], zd1[8] ;

      const Index_t *elemToNode = domain.nodelist(i2);
      Index_t i3=8*i2;
      Real_t volinv=Real_t(1.0)/determ[i2];
      Real_t ss1, mass1, volume13 ;
      for(Index_t i1=0;i1<4;++i1){

         Real_t hourmodx =
            x8n[i3] * gamma[i1][0] + x8n[i3+1] * gamma[i1][1] +
            x8n[i3+2] * gamma[i1][2] + x8n[i3+3] * gamma[i1][3] +
            x8n[i3+4] * gamma[i1][4] + x8n[i3+5] * gamma[i1][5] +
            x8n[i3+6] * gamma[i1][6] + x8n[i3+7] * gamma[i1][7];

         Real_t hourmody =
            y8n[i3] * gamma[i1][0] + y8n[i3+1] * gamma[i1][1] +
            y8n[i3+2] * gamma[i1][2] + y8n[i3+3] * gamma[i1][3] +
            y8n[i3+4] * gamma[i1][4] + y8n[i3+5] * gamma[i1][5] +
            y8n[i3+6] * gamma[i1][6] + y8n[i3+7] * gamma[i1][7];

         Real_t hourmodz =
            z8n[i3] * gamma[i1][0] + z8n[i3+1] * gamma[i1][1] +
            z8n[i3+2] * gamma[i1][2] + z8n[i3+3] * gamma[i1][3] +
            z8n[i3+4] * gamma[i1][4] + z8n[i3+5] * gamma[i1][5] +
            z8n[i3+6] * gamma[i1][6] + z8n[i3+7] * gamma[i1][7];

         hourgam[0][i1] = gamma[i1][0] -  volinv*(dvdx[i3  ] * hourmodx +
                                                  dvdy[i3  ] * hourmody +
                                                  dvdz[i3  ] * hourmodz );

         hourgam[1][i1] = gamma[i1][1] -  volinv*(dvdx[i3+1] * hourmodx +
```

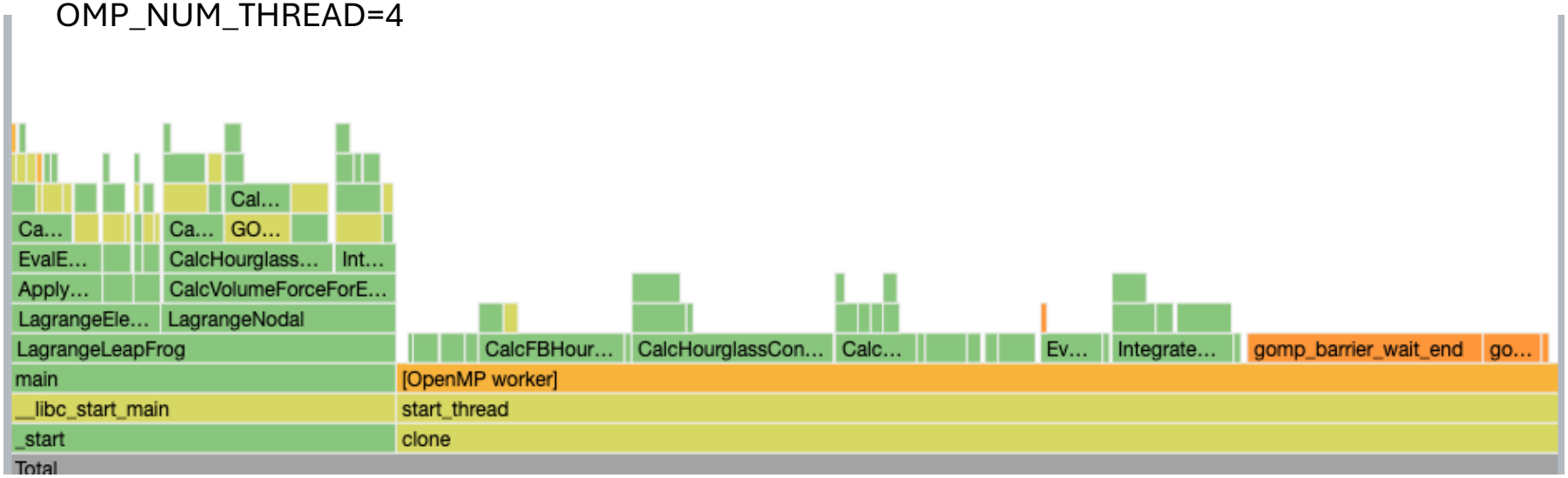
4. SumElemForceFor

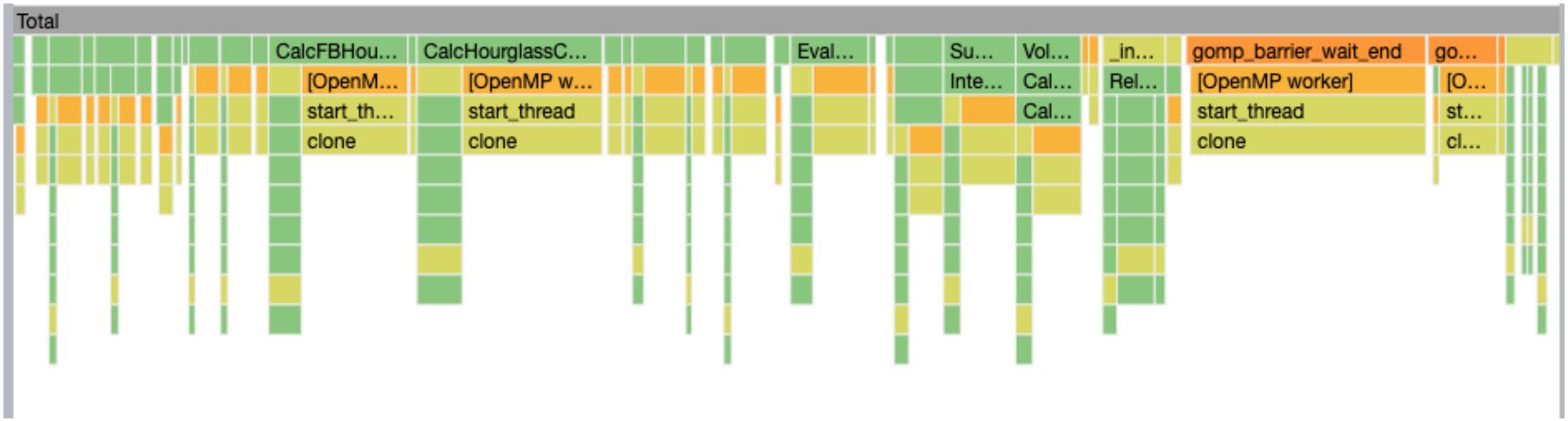
<- CalcElemNodeNormals (7.8%)




```c
static inline
void CalcElemNodeNormals(Real_t pfx[8],
                         Real_t pfy[8],
                         Real_t pfz[8],
                         const Real_t x[8],
                         const Real_t y[8],
                         const Real_t z[8])
{
   for (Index_t i = 0 ; i < 8 ; ++i) {
      pfx[i] = Real_t(0.0);
      pfy[i] = Real_t(0.0);
      pfz[i] = Real_t(0.0);
   }
   /* evaluate face one: nodes 0, 1, 2, 3 */
   SumElemFaceNormal(&pfx[0], &pfy[0], &pfz[0],
                     &pfx[1], &pfy[1], &pfz[1],
                     &pfx[2], &pfy[2], &pfz[2],
                     &pfx[3], &pfy[3], &pfz[3],
                     x[0], y[0], z[0], x[1], y[1], z[1],
                     x[2], y[2], z[2], x[3], y[3], z[3]);
   /* evaluate face two: nodes 0, 4, 5, 1 */
   SumElemFaceNormal(&pfx[0], &pfy[0], &pfz[0],
                     &pfx[4], &pfy[4], &pfz[4],
                     &pfx[5], &pfy[5], &pfz[5],
                     &pfx[1], &pfy[1], &pfz[1],
                     x[0], y[0], z[0], x[4], y[4], z[4],
                     x[5], y[5], z[5], x[1], y[1], z[1]);
   /* evaluate face three: nodes 1, 5, 6, 2 */
   SumElemFaceNormal(&pfx[1], &pfy[1], &pfz[1],
                     &pfx[5], &pfy[5], &pfz[5],
                     &pfx[6], &pfy[6], &pfz[6],
                     &pfx[2], &pfy[2], &pfz[2]
```

OMP_NUM_THREAD=4



Flame Graph 👆

Icicle Graph 👇